

AXEL

Full System Product Requirements Document — v2 Multi-Agent, Multi-Section Automated Trading & Betting System

Prepared: September 20, 2026

Status: Ready-to-build specification, Phase 0–1 scaffold implemented and tested

Important: This document contains hypothetical financial projections built on stated, unproven assumptions (Chapter 9). It is not a guarantee of performance, not a backtest of a real trading strategy, and not financial or legal advice. All strategies referenced here must be validated against real historical data before any capital — paper or real — is put behind them. Automated trading can lose money even when validated.

This document supersedes the v1 build specification. Chapter 16 summarizes every correction made and why.

Contents

1	Executive Summary	5
1.1	What defines this system	5
1.2	What this document contains	5
2	System Architecture — Three Trust Levels	7
2.1	Design Philosophy	7
2.2	The Three Processes	7
2.3	Data Flow for One Decision	9
3	Repository Structure	10
4	Sections — Detailed Specification	12
4.1	Stocks	12
4.1.1	Purpose and edge sources	12
4.1.2	Analysts	12
4.1.3	Candidate strategies for the playbook (pending backtest validation)	12
4.1.4	Execution venue	12
4.2	Options	13
4.2.1	Purpose and edge sources	13
4.2.2	Analysts	13
4.2.3	Candidate strategies for the playbook	13
4.3	Crypto	13
4.3.1	Purpose and edge sources	13
4.3.2	Analysts	13
4.3.3	Candidate strategies for the playbook	14
4.4	Forex	14
4.4.1	Purpose and edge sources	14
4.4.2	Analysts	14
4.4.3	Candidate strategies for the playbook	14
4.5	Bets / Predictions	15
4.5.1	Purpose and edge sources	15
4.5.2	Analysts	15
4.5.3	Candidate strategies for the playbook	15
5	The Quant Research Agent	16
5.1	Purpose	16

5.2	The proposal loop	16
5.3	Hard boundaries (non-negotiable, CI-enforced)	16
6	Risk Management Framework (Corrected)	17
6.1	Layered risk controls	17
6.2	Position sizing — corrected	17
6.3	The kill-switch — defined semantics	18
6.4	Daily stop-loss — corrected	19
6.5	Cross-section correlation	19
6.6	New controls added in v2	19
6.7	Allocation reallocation — corrected	19
7	Data and Machine Learning Infrastructure	20
7.1	Storage	20
7.2	Ingestion	20
7.3	The ML layer	21
7.4	The backtester	21
8	Strategy Validation Methodology — Two Separate Paths	22
8.1	Path 1: Deterministic strategy rules	22
8.1.1	The core rule	22
8.1.2	Why sample size matters	22
8.1.3	Interpreting expectancy realistically	23
8.2	Path 2: LLM Panel judgment	23
9	Financial Projection: \$10,000 Over 6 Months	24
9.1	Assumptions	24
9.2	Capital allocation	24
9.3	Deterministic expected value (no variance)	25
9.4	Monte Carlo simulation — the honest picture	25
9.5	Per-section detail (Base case)	25
9.6	What would move these numbers in reality	26
10	Technology Stack and Language Selection	29
10.1	Mapping languages to trust levels	29
10.2	On Java as an alternative	29
10.3	Reference open-source repositories	30
11	Accounts, Capital, and Security	31
11.1	Account structure	31
11.2	Security requirements (non-negotiable)	31
11.3	Self-custody wallets vs. exchange accounts (Crypto)	31
12	Testing Strategy	32
12.1	Priority order	32
13	Phased Build Plan	33
14	Validation and Promotion Gates	35

14.1 Strategy admission to the playbook	35
14.2 Validating the LLM layer	35
14.3 Paper-to-live gate (per section)	35
15 Top Risks and Mitigations	37
16 Summary: Every Correction from v1 to v2	38

List of Figures

2.1	Axel's three trust levels. Every arrow crossing a boundary is a typed Postgres record. The CI lint check <code>infra/ci/check_no_llm_imports.py</code> enforces, at build time, that the Trading core and Watchdog import nothing from the Agent service or any LLM SDK.	8
9.1	Starting capital allocation across Axel's five sections.	27
9.2	Simulated 6-month equity curve, Base case, 500 of 2,000 simulated paths shown for texture, with 25th–75th and 5th–95th percentile bands.	27
9.3	Distribution of month-6 outcomes across all three scenarios.	28
9.4	Median simulated outcome per section after 6 months, Base case, versus starting capital.	28

1. Executive Summary

Axel is a hierarchical, multi-agent system designed to evaluate and execute trading and betting decisions across five independent market sections — Stocks, Options, Crypto, Forex, and Bets/Predictions — under a single prime overseer agent, Axel. This v2 specification corrects a set of architectural and risk-math issues identified in an implementation review of the v1 design, and adds the operational detail (testing strategy, phased build plan, validation gates) needed to actually start building.

1.1 What defines this system

1. **Three trust levels, strictly separated.** An *Agent service* (LLM-driven, untrusted/creative) proposes actions. A *Trading core* (deterministic, CI-enforced to import no LLM code) validates, sizes, and can veto every proposal. A *Watchdog* (independent process, separate DB role) runs the kill-switch and can halt the system even if the Trading core itself has failed. **Axel’s LLM half proposes; it does not dispose** — this is the single most important correction from v1, detailed in Chapter 2.
2. **Nothing is trusted with capital until validated** — and validated the *right way* for what it is. Deterministic strategy rules are validated by backtesting against real historical data. LLM judgment cannot be validated this way (a model can have memorized the backtest period) and is instead validated forward, in paper trading, with calibration tracking. This distinction, absent from v1, is detailed in Chapter 7.
3. **Every decision is logged, typed, and replayable.** Every record that crosses a trust boundary is a versioned Pydantic contract written to Postgres/TimescaleDB — never a direct function call between untrusted and trusted code.

1.2 What this document contains

- The corrected three-process architecture and full repository structure, as actually implemented and tested (Chapters 2–3)
- Each section’s internal design in depth: Stocks, Options, Crypto, Forex, Bets/Predictions (Chapter 4)
- The Quant Research Agent’s proposal-only improvement loop (Chapter 5)
- The corrected risk management framework: clarified Kelly sizing, risk-at-stop caps, kill-switch semantics, per-section loss policies (Chapter 6)

- Data and machine learning infrastructure (Chapter 7)
- Strategy validation methodology, split for deterministic rules vs. LLM judgment (Chapter 8)
- A financial projection for \$10,000 over 6 months with full Monte Carlo uncertainty bands (Chapter 9)
- Technology stack and language selection, mapped to the three trust levels (Chapter 10)
- Accounts, capital, and security requirements (Chapter 11)
- Testing strategy, including the property-based test suite that is the project's most important safety net (Chapter 12)
- A phased, vertical build plan with explicit exit criteria per phase (Chapter 13)
- Validation and promotion gates with fixed, pre-committed thresholds (Chapter 14)
- Top risks and mitigations (Chapter 15)
- A full summary of every correction made from v1 to v2, and why (Chapter 16)

Design Note

Everything in this document has a corresponding line of code. The risk package (Chapter 6) and its property-test suite are real, tested, and passing today — not aspirational. Everything upstream of the risk boundary (analysts, data ingestion, live broker wiring) is intentionally still a stub, per the build order in Chapter 13: the deterministic safety layer must exist and be proven *before* anything is allowed to send it an order.

2. System Architecture — Three Trust Levels

2.1 Design Philosophy

Four principles govern every design decision in Axel v2:

1. **Reasoning and execution are architecturally separated, and enforced, not just documented.** Any component that calls an LLM may be exploratory, creative, or wrong without immediate financial consequence. Any component that touches capital is deterministic, auditable code with hard-coded limits the reasoning layer cannot override — and a CI lint check fails the build if a deterministic module ever imports an LLM library.
2. **The boundary between trust levels is a typed database record, never a function call.** The Agent service writes `Signal`, `Proposal`, and `AllocationProposal` rows to Postgres. The Trading core reads them. There is no code path by which Agent-service logic can directly invoke execution logic.
3. **Nothing is trusted with capital until it is validated the correct way for what it is.** Deterministic rules are backtested. LLM judgment is validated forward, in paper trading, with calibration tracking — never backtested against a period the model may have seen in training.
4. **Every decision is logged and explainable, and the log itself is tamper-evident.** Append-only tables (kill-switch events, approvals, audit log) have no `UPDATE/DELETE` grants for the application role.

2.2 The Three Processes

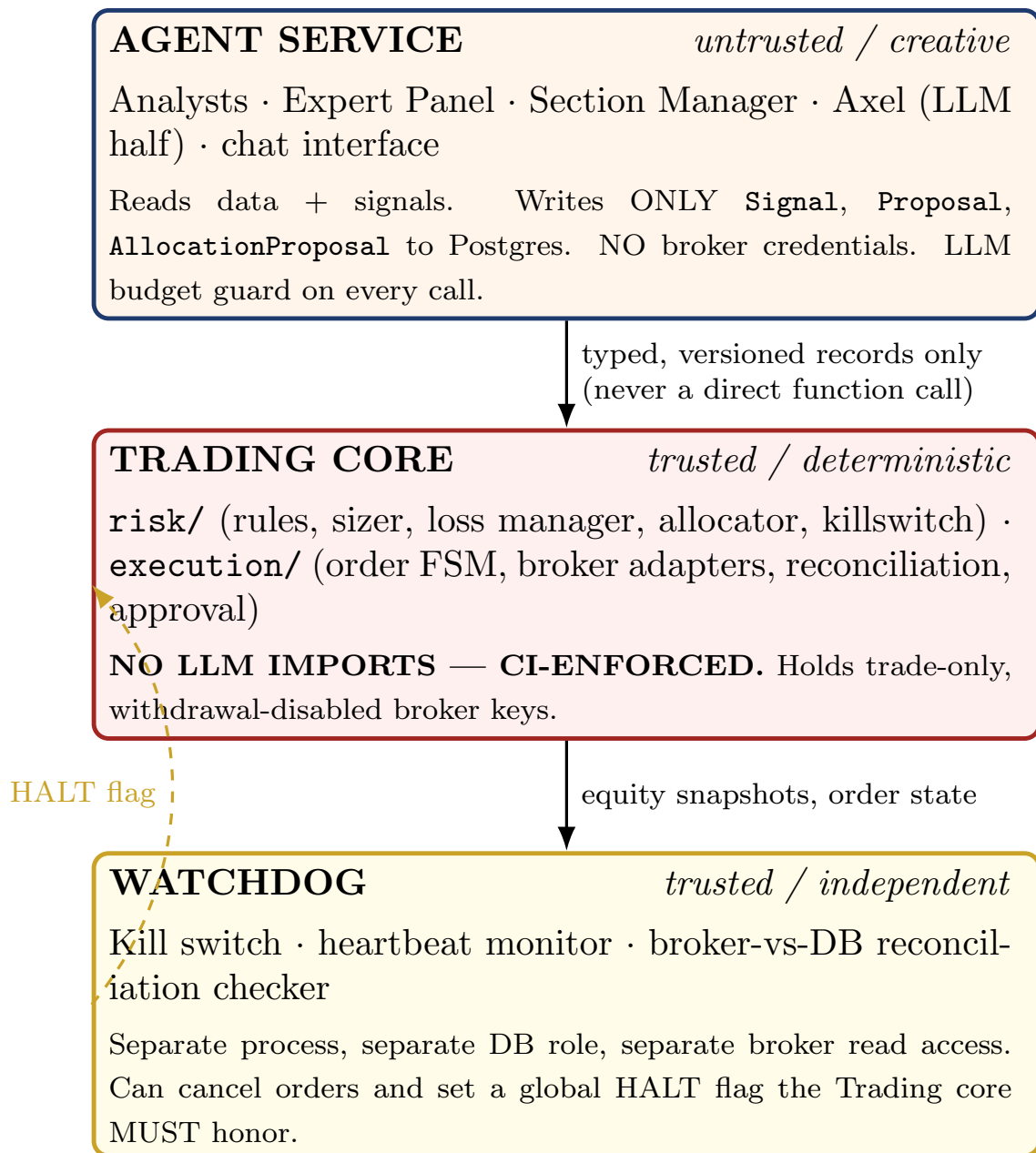
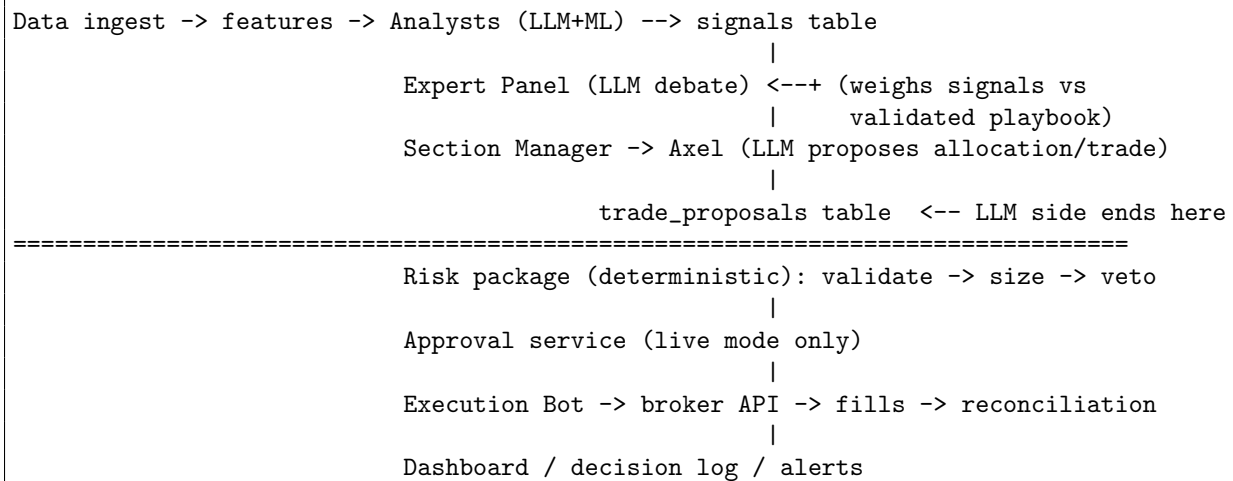


Figure 2.1: Axel's three trust levels. Every arrow crossing a boundary is a typed Postgres record. The CI lint check `infra/ci/check_no_llm_imports.py` enforces, at build time, that the Trading core and Watchdog import nothing from the Agent service or any LLM SDK.

2.3 Data Flow for One Decision



Corrected in v2

v1 had Axel described as an "LLM + deterministic hybrid" that also served as the "final trade gate." That is a contradiction: an LLM in the last gate breaks the entire reasoning/execution separation the rest of the design depends on. In v2, **Axel's LLM half only emits a typed AllocationProposal**. The actual final gate is the deterministic Risk package below the dotted line above — it validates, sizes, and can veto every proposal Axel or any Section Manager emits, with zero exceptions.

3. Repository Structure

The structure below matches the repository as actually scaffolded and tested for this specification — not an aspirational diagram. The risk package and its property-test suite are working code; modules marked [stub] are intentional stubs (`NotImplementedError`) pending the build phases in Chapter 13.

```
axel_v2/
|-- core/                                # shared by every process
|   |-- types.py                        # Signal, Proposal, RiskDecision, Order,
|   |                                   # AllocationProposal (pydantic contracts)
|   |-- clock.py                        # injectable Clock / FakeClock (deterministic replay)
|   |-- ids.py                          # idempotent client_order_id generation
|   \-- config.py                       # RiskConfig -- thresholds fixed BEFORE seeing results
|
|-- risk/                                # DETERMINISTIC. No LLM imports (CI-enforced). [WORKING]
|   |-- sizer.py                        # Kelly f* -> fractional Kelly -> min(caps)
|   |-- rules.py                        # gross exposure, correlation, leverage, data hygiene
|   |-- loss_manager.py                 # daily stop-loss, per-section flatten policy
|   |-- allocator.py                   # clamps reallocation proposals, logs rejected excess
|   \-- killswitch.py                  # peak-to-trough drawdown vs high-water mark
|
|-- execution/                           # DETERMINISTIC. No LLM imports (CI-enforced).
|   |-- broker_base.py                  # abstract adapter interface
|   |-- alpaca.py [stub]                # Alpaca paper/live adapter
|   |-- order_fsm.py                    # valid state transitions only [WORKING]
|   |-- reconcile.py                    # broker-vs-DB mismatch detection
|   \-- approval.py                    # human-approval gate with expiry
|
|-- watchdog/                            # INDEPENDENT trust level. No LLM imports (CI-enforced).
|   |-- watchdog.py [stub]              # main loop: equity snapshot -> killswitch -> halt
|   \-- heartbeat.py                   # process-silence detection
|
|-- agents/                              # UNTRUSTED / CREATIVE. LLM side. (mostly [stub])
|   |-- llm_budget.py                   # per-hour/per-day spend guard [WORKING]
|   |-- analysts/base_analyst.py        # untrusted-input handling contract
|   |-- panel.py                        # weighs signals vs validated playbook only
|   |-- section_manager.py              # Panel decision -> draft Proposal
|   \-- axel.py                         # proposes allocation shifts only -- NOT the final gate
|
|-- sections/                            # one folder per asset class, identical substructure
|   |-- stocks/                         {config.yaml, playbook/, analysts/, risk/overrides.py}
|   |-- options/                        {config.yaml, playbook/, analysts/, risk/overrides.py}
|   |-- crypto/                         {config.yaml, playbook/, analysts/, risk/overrides.py}
|   |-- forex/                          {config.yaml, playbook/, analysts/, risk/overrides.py}
|   \-- bets_predictions/ {config.yaml, playbook/, analysts/, risk/overrides.py}
|
|-- quant_research/                      # proposal-only; PR-based; cannot touch risk/ or execution/
|   |-- quant_agent.py [stub]
|   \-- proposals/
|
|-- data/
|   |-- ingest/ [stub]                  # market_data, news_sentiment, email_reader,
```

```

| |                                     # telegram_signals [log-only], copy_trade_feeds [advisory]
| |-- features/ [stub]                 # point-in-time feature engineering
| \-- ml/
| |   |-- backtester.py [stub]         # fees, slippage, walk-forward split
| |   \-- validation.py                # strategy admission gate -- fixed thresholds [WORKING]
|
|-- comms/ [stub]                      # email_in, email_out, chat (read-only at first)
|-- dashboard/grafana/                # Grafana provisioning (default)
|
|-- infra/
| |   |-- docker-compose.yml           # three services, three DB roles, matching trust levels
| |   |-- migrations/001_init.sql      # full Postgres/TimescaleDB schema
| |   |-- ci/check_no_llm_imports.py  # [WORKING, TESTED -- catches real violations]
| |   \-- .env.example
|
\-- tests/
    |-- property/test_risk_invariants.py # Hypothesis -- highest-priority suite [8/8 PASS]
    |-- unit/                           # loss manager, order FSM [PASS]
    |-- replay/                         # deterministic pipeline replay (planned)
    \-- integration/                   # broker contracts, chaos drills (planned)

```

Design Note

Build status, verified. Running `pytest tests/property/ tests/unit/ -v` against this repository passes 8/8 tests, including a property-based check that no approved order size can exceed any configured cap under any randomized input. The CI lint check has been verified to both pass on clean code and fail with exit code 1 when an LLM import is deliberately introduced into `risk/`.

4. Sections — Detailed Specification

Every section shares the same internal chain (Analysts → Expert Panel → Section Risk overrides layered on the core Risk package → Execution), but the analysts, data sources, and applicable strategies differ meaningfully by asset class. Each is detailed below. This chapter is unchanged in substance from v1; the correction in v2 is architectural (Chapters 2, 6), not sectional.

4.1 Stocks

4.1.1 Purpose and edge sources

Equities are Axel’s most mature section: liquidity is high, free paper-trading APIs exist, and the strategy literature is the deepest of any asset class covered here. Edge sources with published, peer-reviewed backing include cross-sectional momentum, the low-volatility anomaly, and factor exposures (value, size, profitability) documented in the Fama–French factor library.

4.1.2 Analysts

- **Technical Analyst** — price/volume pattern recognition (moving average crossovers, RSI, volatility breakouts).
- **Fundamental Analyst** — parses SEC filings (10-K/10-Q/8-K via EDGAR), earnings surprises, balance-sheet ratios.
- **Sentiment Analyst** — news and social sentiment scoring (NewsAPI, GDELT, FinGPT-based scoring).
- **Macro Analyst** — rate environment, inflation prints, employment data (FRED series) as a regime filter.

4.1.3 Candidate strategies for the playbook (pending backtest validation)

- Cross-sectional momentum (12-1 month formation)
- Low-volatility anomaly (long low-beta, short/avoid high-beta within a sector-neutral basket)
- Mean-reversion pairs trading on cointegrated equity pairs

4.1.4 Execution venue

Alpaca (paper → live). Options overlays route through the same broker where supported.

4.2 Options

4.2.1 Purpose and edge sources

Options edge is mathematically distinct from directional equity edge: the primary documented anomaly is the **variance risk premium** — implied volatility tends to run above subsequently realized volatility.

4.2.2 Analysts

- **Volatility Analyst** — implied vs. realized volatility spread, term structure shape.
- **Greeks Analyst** — delta/gamma/theta/vega exposure of candidate positions (via the `py_vollib` and `QuantLib` libraries).
- **Sentiment Analyst** — shared logic with the Stocks section’s sentiment analyst, re-scoped to the underlying.
- **Macro Analyst** — rate environment (affects option pricing directly via the risk-free rate term).

4.2.3 Candidate strategies for the playbook

- Short-dated covered calls / cash-secured puts on the Stocks section’s core holdings (income overlay)
- Iron condors when implied volatility is elevated relative to its own historical percentile
- Calendar spreads to harvest term-structure mispricing

Risk Note

Options strategies that sell volatility have asymmetric tail risk — small regular gains, occasional large losses in a volatility spike. Per Chapter 6, Options is restricted to **defined-risk structures only** until proven, and carries a tighter position cap than the general 5% section limit.

4.3 Crypto

4.3.1 Purpose and edge sources

Crypto is the most API-friendly section (24/7 markets, mature open-source bot ecosystem) and the highest-frequency by trade count in this specification. Its distinctive, mechanically-driven edge is **funding-rate arbitrage** on perpetual futures.

4.3.2 Analysts

- **Technical Analyst** — same pattern-recognition approach as Stocks, tuned for crypto’s higher volatility.
- **On-chain Analyst** — exchange inflow/outflow, whale wallet movement.
- **Sentiment Analyst** — crypto-specific social sentiment (higher retail noise than equities; weighted accordingly).

- **Funding-Rate Analyst** — perpetual futures funding rate level and trend across exchanges.

4.3.3 Candidate strategies for the playbook

- Funding-rate arbitrage (long spot / short perpetual, collecting the funding payment while price-neutral)
- Cross-exchange price arbitrage where latency permits
- On-chain flow-based positioning

Corrected in v2

Per Chapter 6, Crypto operates under a strict **no-leverage policy** and its daily stop-loss check runs on a rolling 24-hour UTC window rather than a calendar-day/exchange-local boundary, since the market never closes.

4.4 Forex

4.4.1 Purpose and edge sources

Forex is the most macro-driven section. Its best-documented edge is the **carry trade** and reaction to scheduled macro releases.

4.4.2 Analysts

- **Technical Analyst** — trend and range identification on major pairs.
- **Rate-Differential Analyst** — tracks interest-rate spreads between currency pairs.
- **Carry-Trade Analyst** — identifies and sizes carry opportunities, net of volatility-adjusted risk.
- **Macro Analyst** — economic calendar awareness (NFP, CPI, central bank meetings).

4.4.3 Candidate strategies for the playbook

- Carry trade on G10 currency pairs with volatility-adjusted position sizing
- Purchasing power parity mean-reversion over long horizons
- Momentum around scheduled macro releases (with an execution delay to avoid front-running noise)

Risk Note

Carry trades earn a small, steady premium most of the time and then suffer a sharp loss during a volatility spike or unwind. The Forex Section Risk override specifically monitors realized FX volatility as an early-warning input.

4.5 Bets / Predictions

4.5.1 Purpose and edge sources

This section covers regulated prediction markets (Kalshi) as the primary venue, with sports-odds arbitrage as a secondary, math-driven strategy.

4.5.2 Analysts

- **Odds-Movement Analyst** — tracks line/price movement across the contract's life or across sportsbooks.
- **News Analyst** — event-relevant news that should move a contract's fair value.
- **Calibration Analyst** — compares a contract's implied probability against a model-based fair probability.
- **Line-Value Analyst** — for sports odds, tracks closing-line value as the standard proof of a real edge.

4.5.3 Candidate strategies for the playbook

- Cross-book arbitrage on sports odds (mechanical, not predictive — lowest-risk candidate in this section)
- Near-expiry mispricing on low-liquidity Kalshi contracts
- Event-driven positioning ahead of scheduled data releases

Risk Note

Legality of prediction-market and sports-odds venues varies by jurisdiction and platform. This must be confirmed per venue before any execution bot in this section is connected to a funded account — see Phase 9 in Chapter 13, which is explicitly gated on written legal review.

5. The Quant Research Agent

5.1 Purpose

The Quant Research Agent (QRA) is Axel’s mechanism for continuous improvement: it watches each section’s live and paper-trading performance, identifies candidate strategy or parameter changes, and proposes them. It is architecturally barred from deploying anything on its own, and lives inside the Agent service’s trust level even though its output is code rather than a trade.

5.2 The proposal loop

1. **Observe (read-only).** Reads recent performance metrics — expectancy (R-multiple), Sharpe ratio, drawdown — from Postgres/TimescaleDB.
2. **Propose.** Writes a candidate as a diff/patch file under `quant_research/proposals/`, never directly into a section’s live files.
3. **Backtest.** Runs the candidate through `data/ml/backtester.py` against real historical data, out-of-sample.
4. **Open a pull request.** If the candidate clears the improvement threshold (0.05R minimum), opens a PR with the diff and backtest metrics attached.
5. **Human review and merge. The QRA never merges its own PR.**
6. **Paper-trading validation.** Even after merge, the change must pass the same paper-trading validation phase as any other strategy (Chapter 8) before it touches real capital.

5.3 Hard boundaries (non-negotiable, CI-enforced)

- Never merges its own pull requests.
- Never edits any file under `risk/`, `execution/`, or `watchdog/` — enforced by the same CI lint check that governs LLM imports generally (`infra/ci/check_no_llm_imports.py`).
- Cannot edit its own source (`quant_research/quant_agent.py`).
- Proposals with fewer than a statistically meaningful number of out-of-sample trades are not eligible to be surfaced as a PR.

6. Risk Management Framework (Corrected)

This chapter replaces the v1 risk table wholesale. v1's numbers were individually reasonable but collided with each other in ways that only surface once you try to implement them — this chapter is written to be directly implementable, and in fact already has been (see **risk/** in Chapter 3).

6.1 Layered risk controls

Level	Scope	Mechanism
Position-level	Single trade	Fractional-Kelly sizing, THEN clamped by both a notional cap and a risk-at-stop cap — see Section 6.2
Section-level	One asset class	Daily stop-loss on realized+unrealized P&L since section-day-start; explicit per-section flatten policy on breach
Portfolio-level	Entire capital across all sections	10% peak-to-trough drawdown kill-switch (vs. high-water mark, not starting capital); max 5% capital reallocation per cycle; cross-section correlation monitoring

6.2 Position sizing — corrected

Corrected in v2

v1 ambiguity: "Kelly capped at 25%" versus "max position 5%" collided — unclear whether 25% meant 25% of Kelly or 25% of capital, and if the latter, the 5% cap always won and the Kelly math did nothing. **v2 fix:** 25% always means $0.25 \times \text{full Kelly } f^*$, and that result is then run through two separate, independent caps.

$$f^* = \frac{b \cdot p - q}{b}, \quad q = 1 - p$$

$$\text{approved size} = \min \left(\underbrace{0.25 \cdot f^* \cdot C}_{\text{fractional Kelly}}, \underbrace{0.05 \cdot C}_{\text{5\% notional cap}}, \underbrace{\frac{0.01 \cdot C}{\text{stop distance \%}}}_{\text{1\% risk-at-stop cap}} \right)$$

where C is the section's current capital, p is win probability, b is the win/loss payoff ratio, and stop distance % is the fractional distance between entry and stop price.

Corrected in v2

v1 gap: the 5% notional cap alone did not account for stop distance — a 5% notional position with a wide stop could lose far more than intended, and the 3% daily stop was loose against that. **v2 fix:** added the **risk-at-stop cap** above, computed from the actual recorded stop distance at entry. Every sizing decision logs which of the three terms was binding (`risk/sizer.py`).

If $f^* \leq 0$ (non-positive estimated edge), size is zero, full stop — no amount of Kelly math justifies sizing a negative-expectancy trade. Estimates of p and b come from validated out-of-sample backtest and paper-trading history, with a **haircut applied** (e.g. using the lower bound of a confidence interval), because an overestimated edge makes Kelly dangerous rather than optimal.

Design Note

This sizing function is implemented and tested. `tests/property/test_risk_invariants.py` asserts, via Hypothesis property-based testing across randomized inputs, that the approved size can *never* exceed either cap regardless of how favorable the win-probability and payoff-ratio inputs look. This is the single most valuable test in the project.

6.3 The kill-switch — defined semantics

Corrected in v2

v1 gap: "pause" was undefined. Open options positions and 24/7 crypto keep moving during an undefined pause. **v2 fix:** full definition below, and the kill-switch now runs inside an **independent Watchdog process** (Chapter 2), not inside the Trading core it is meant to check.

Drawdown definition: peak-to-trough on total marked-to-market equity, measured against the running **high-water mark** — not starting capital. A system that has grown 50% and then drawn down 10% from its peak still halts, because the risk of continuing is about the trajectory, not the absolute starting point.

On trigger (10% drawdown from high-water mark): the Watchdog sets a global **HALT** flag that the Trading core must honor before evaluating any further proposal. Per-section actions on halt follow an **explicit, documented flatten policy** rather than a uniform default:

Section	Flatten policy on breach
Stocks	Cancel working orders, hold existing positions
Options	Cancel working orders, hold existing positions (early close risks unwanted assignment)
Crypto	Cancel working orders, flatten positions (24/7 market, no overnight-gap rationale to hold)
Forex	Cancel working orders, hold existing positions

Section	Flatten policy on breach
Bets/Predictions	Cancel working orders, hold existing positions (contracts are typically illiquid to unwind early)

Resume: manual only, with a written reason logged to the append-only `killswitch_events` table. There is no automatic resume condition.

6.4 Daily stop-loss — corrected

Measured on **realized + unrealized** P&L, since section-day-start. Day boundary is **exchange-local** for Stocks/Options/Forex, and **rolling 24-hour UTC** for Crypto specifically, since a 24/7 market has no natural exchange-local boundary. On breach: block new entries; cancel working orders; flatten per the table above.

6.5 Cross-section correlation

The Risk Engine monitors realized correlation between sections' recent returns and flags/reduces exposure if correlation exceeds 0.7 — at that point, the "multiple independent sections" assumption underlying the portfolio-level risk model has broken down, most commonly during a broad risk-off macro event.

6.6 New controls added in v2

- **Gross exposure caps**, per-section and total, in addition to the position-level caps above.
- **No-leverage policy for Crypto**, checked on every proposal.
- **Defined-risk-structures-only for Options**, until the section has a proven track record.
- **Symbol/sector concentration cap** across the whole book, since Axel can see total exposure across all five sections even where individual Section Managers cannot.
- **Data and order hygiene checks:** reject on stale quotes, market closed, unusual spread, order rate above limit, or a duplicate `client_order_id` (idempotency).

6.7 Allocation reallocation — corrected

Corrected in v2

v1 gap: the 5% max-reallocation-per-cycle rule had no defined behavior for what happens to a request above that limit. **v2 fix:** the excess above the cap is **explicitly rejected and logged**, not silently scaled down — the distinction matters for audit, since a silently-scaled request looks identical in the log to Axel having asked for less than it actually did.

7. Data and Machine Learning Infrastructure

7.1 Storage

PostgreSQL, extended with TimescaleDB for time-series hypertables, is the system of record. Key tables, per the schema actually implemented in `infra/migrations/001_init.sql`:

Table	Type	Purpose
instruments, calendars	regular	Symbols, venues, trading hours, holidays, corporate actions
bars, quotes, funding_rates	hypertable	Market data, stored with both event time and ingestion time (point-in-time correctness)
news_items, filings	hypertable	Raw ingested text with source, trust flag, and hash for dedupe
signals	hypertable	Analyst outputs: schema-validated, with model and prompt version
panel_decisions, trade_proposals	regular	Panel rationale, confidence, and the proposal handed to the core
risk_decisions	regular	Every approval and veto with the binding limit and reasons
orders, fills, positions_snapshots	regular/ hypertable	Order state machine history, executions, periodic position snapshots
equity_snapshots	hypertable	Per-section and total equity; feeds drawdown and dashboard
strategy_registry	regular	Status: candidate / validated / paper / live; link to backtest report and parameters hash
agent_runs	regular	Model, prompt version, tokens, cost, latency, input/output refs for every LLM call
killswitch_events, approvals, audit_log	append-only	No UPDATE/DELETE grants for the application role

7.2 Ingestion

Source	Handling
Market data	Direct API pull (Alpaca, Polygon.io, exchange APIs, OANDA), point-in-time

Source	Handling
News/sentiment	NewsAPI/GDELT pulled and scored, feeding the relevant Sentiment Analyst
Email (inbound)	Parsed via Gmail API for broker alerts, newsletters, forwarded signals
Telegram signals	Logged only by default. Never wired directly to execution. A channel's historical accuracy must be backtested before it is even treated as advisory input to any Expert Panel
Copy-trade feeds	Only from platforms with a verifiable, audited track record. Treated as one advisory signal among several — never a direct-to-execution pipe

Risk Note

Prompt injection. All ingested text — email, news, social, Telegram — is treated as **untrusted data, never as instructions**. Analysts have no side-effect tools and must return schema-validated `Signal JSON (core.types.Signal)`. This is enforced structurally in `agents/analysts/base_analyst.py`, and tested via adversarial prompt tests (Chapter 12) that inject strings into fake emails/headlines and assert no Proposal's action or size changes as a result.

7.3 The ML layer

Separate from the LLM-driven Analysts and Expert Panel, the ML layer trains and runs conventional models (scikit-learn, gradient-boosted trees via LightGBM/XGBoost first; reinforcement learning via FinRL deferred until a simpler baseline is beaten and validated) on historical data to generate quantitative signals. This keeps "the math" out of the LLM's hands — an LLM asked to compute statistics on the fly risks silent arithmetic errors that a tested, versioned model function will not.

7.4 The backtester

`data/ml/backtester.py` is the gate between "a strategy an LLM read about in a paper" and "a strategy Axel is allowed to trust with capital." It must support walk-forward (out-of-sample) testing with an untouched final holdout block, transaction cost and slippage modeling, market-calendar awareness, and a multiple-testing adjustment (deflated Sharpe) when many strategy variants have been tried.

8. Strategy Validation Methodology — Two Separate Paths

Corrected in v2

The single most important correction in this document. v1 implied that "backtested and scored" applied uniformly to every strategy, including the LLM Panel's own judgment. This is unsound: an LLM trained on data including the historical backtest period can effectively *recall* what happened rather than demonstrate genuine skill, contaminating any backtest of its own decisions. **v2 splits validation into two separate paths, below.**

8.1 Path 1: Deterministic strategy rules

Moving-average crossovers, carry-trade sizing formulas, funding-rate arbitrage thresholds — anything expressible as code with no memory of training data — is validated by backtesting against real historical data, out-of-sample, per the fixed thresholds in Chapter 14. This works precisely because the rule has no capacity to have "seen" the test period in any sense that matters.

8.1.1 The core rule

No strategy enters a section's **playbook/** — and is therefore never weighed by an Expert Panel in a live decision — until it has:

1. Been backtested against real historical data (never synthetic)
2. Been evaluated out-of-sample, on data not used to design or tune it
3. Cleared a minimum sample size (at least 100 out-of-sample trades)
4. Been scored on expectancy and risk-adjusted return — not win rate alone
5. Survived a robustness check: $\pm 20\%$ parameter perturbation does not flip the sign of expectancy

8.1.2 Why sample size matters

A 70% win rate over 20 trades is not evidence of a 70% edge — it is well within the range of outcomes a coin flip can produce by chance over a small sample. Strategies that appear only in a small backtest window are flagged as *unproven*, not *validated*, in the **strategy_registry** table until they accumulate enough trade history.

8.1.3 Interpreting expectancy realistically

$$\text{Expectancy (R)} = (\text{Win}\% \times \text{Avg Win in R}) - (\text{Loss}\% \times \text{Avg Loss in R})$$

Expectancy range	Interpretation	Note
Negative 0.0R–0.1R	Losing strategy Marginal	Discard regardless of win rate Likely not worth transaction costs at moderate trade frequency
0.1R–0.5R	Realistic, professional-grade	The believable target range for a genuinely validated strategy
Above 1R, sustained	Treat with suspicion	Almost certainly overfitting, leakage, or a data error until proven otherwise over a very large out-of-sample set

8.2 Path 2: LLM Panel judgment

The Expert Panel’s weighing of signals cannot be backtested the way a deterministic rule can. Instead:

1. **Validated forward, in paper trading only** — never retrospectively against a period the model may have seen in training.
2. **Calibration tracking.** Panel confidence is treated as a feature to be measured, not trusted at face value. Track a Brier score and reliability curve against actual paper-trading outcomes; **re-derive the minimum-confidence threshold from that data**, replacing the 0.3 placeholder inherited from v1.
3. **Ablation against a rules-only baseline.** Run a deterministic-only baseline in parallel. If the LLM Panel does not beat it, net of costs, over the soak period — simplify. This is a real, acceptable outcome, not a failure of the exercise.
4. **Track analyst disagreement** as an additional signal of when the Panel’s confidence is or isn’t well-founded.

Risk Note

This table is the most important sanity check for Path 1, and the ablation test is the equivalent check for Path 2. If a backtest — from any section, the Quant Research Agent, or a public repository — reports expectancy far outside 0.1R–0.5R, look harder for a bug before getting excited. If the LLM Panel never beats the rules-only baseline after costs, that is real information, not a reason to keep tuning prompts indefinitely.

9. Financial Projection: \$10,000 Over 6 Months

Risk Note

This entire chapter is a hypothetical model built on stated, unproven assumptions. None of the strategies described in Chapter 4 have yet been validated per Chapter 8's methodology. The win rate and R-multiples used below are illustrative planning inputs, not measured results. Treat every number here as "if the system performs this well, here is what that would look like," not as a prediction. Past backtest or simulated performance never guarantees future results, and all trading and betting activity carries a real risk of loss, including loss of the full amount deployed. This is not financial advice.

9.1 Assumptions

Parameter	Value
Starting capital	\$10,000
Horizon	6 months
Risk per trade	1% of that section's current capital (compounding)
Assumed win probability (p)	45% across all sections (a deliberately modest, believable assumption)
Base R-multiple (expectancy)	Section-specific, 0.15R–0.25R — within the realistic professional range from Chapter 8
Win/loss payoff ratio (b)	Solved per section so that p and base R together produce the stated expectancy
Scenarios modeled	Conservative ($R \times 0.5$), Base ($R \times 1.0$), Optimistic ($R \times 1.5$)
Simulation method	Monte Carlo, 2,000 independent paths per scenario, trade-level compounding

9.2 Capital allocation

Section	Alloc %	Starting \$	Trades/mo	Base R
Stocks	30%	\$3,000	15	0.20
Options	10%	\$1,000	8	0.15

Section	Alloc %	Starting \$	Trades/mo	Base R
Crypto	25%	\$2,500	30	0.25
Forex	15%	\$1,500	12	0.15
Bets/Predictions	20%	\$2,000	10	0.20
Total	100%	\$10,000	—	—

9.3 Deterministic expected value (no variance)

Scenario	Month-6 capital	Return
Conservative (R $\times$ 0.5)	\$11,156	+11.6%
Base (R $\times$ 1.0)	\$12,510	+25.1%
Optimistic (R $\times$ 1.5)	\$14,103	+41.0%

Design Note

This table alone is misleading in isolation — it shows only the average path, not the range of real outcomes genuine trade-level variance would produce. The Monte Carlo simulation below shows the same scenarios with uncertainty bands.

9.4 Monte Carlo simulation — the honest picture

Scenario	5th pct	25th pct	Median	75th pct	95th pct	P(loss)
Conservative	\$10,058	\$10,641	\$11,126	\$11,589	\$12,344	4.0%
Base	\$11,181	\$11,902	\$12,489	\$13,073	\$14,087	0.1%
Optimistic	\$12,304	\$13,237	\$14,025	\$14,898	\$16,212	0.0%

Reading this table correctly: under the Base case, half of simulated outcomes land above \$12,489 and half below — and 5% of simulated paths land below \$11,181 despite the positive expectancy built into every trade. The $P(loss)$ column is low in this model *specifically because* the assumed edge is real and consistently present across every trade in the simulation. In live deployment, that is precisely the assumption Chapter 8’s validation process exists to test before it is earned.

9.5 Per-section detail (Base case)

Section	Start	Median month 6	@ Median return	re-
Stocks	\$3,000	\$3,611	+20.4%	
Options	\$1,000	\$1,081	+8.1%	
Crypto	\$2,500	\$3,852	+54.1%	
Forex	\$1,500	\$1,687	+12.5%	

Section	Start	Median month 6	@	Median turn	re-
Bets/Predictions	\$2,000	\$2,243		+12.1%	

Crypto shows the largest median gain here purely because it is assigned the highest trade frequency and highest base R among the five sections — both planning assumptions, not measured results.

9.6 What would move these numbers in reality

- **Transaction costs and slippage are not modeled above.** At the lower end of the assumed R range, fees alone could remove a meaningful fraction of the edge, especially in higher-frequency sections.
- **The 45% win-probability assumption is a planning input, not a measurement.** Real validated strategies may show a different p/b combination entirely.
- **Correlation between sections is assumed at zero here.** Chapter 6's correlation monitoring exists because real markets do not always honor that assumption.
- **Capital reallocation by the allocator is not modeled here** — the simulation holds each section's allocation fixed for 6 months, whereas Axel's real behavior may shift capital toward better performers over time, clamped to 5% per cycle.

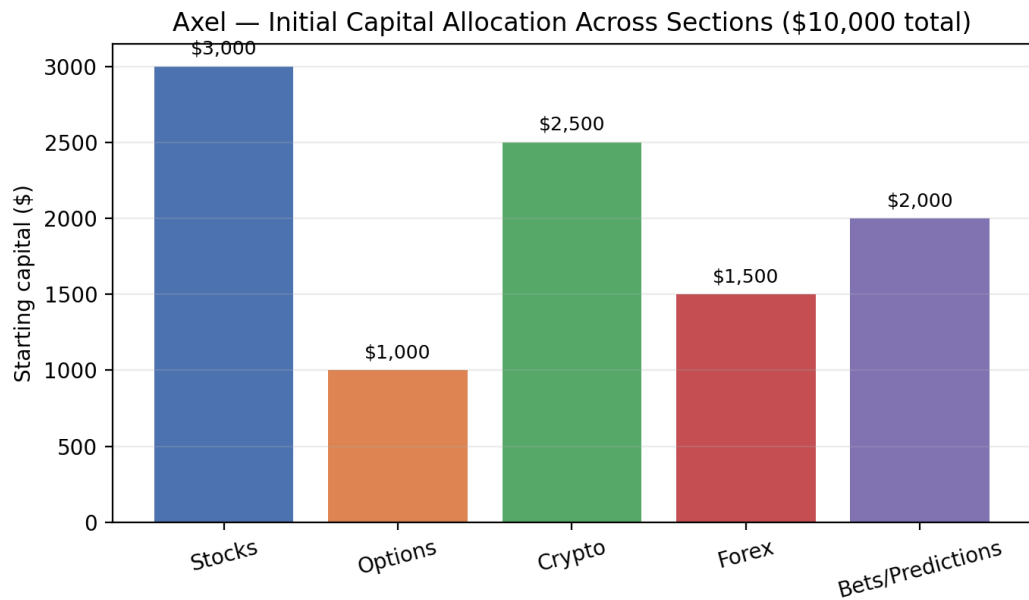


Figure 9.1: Starting capital allocation across Axel’s five sections.

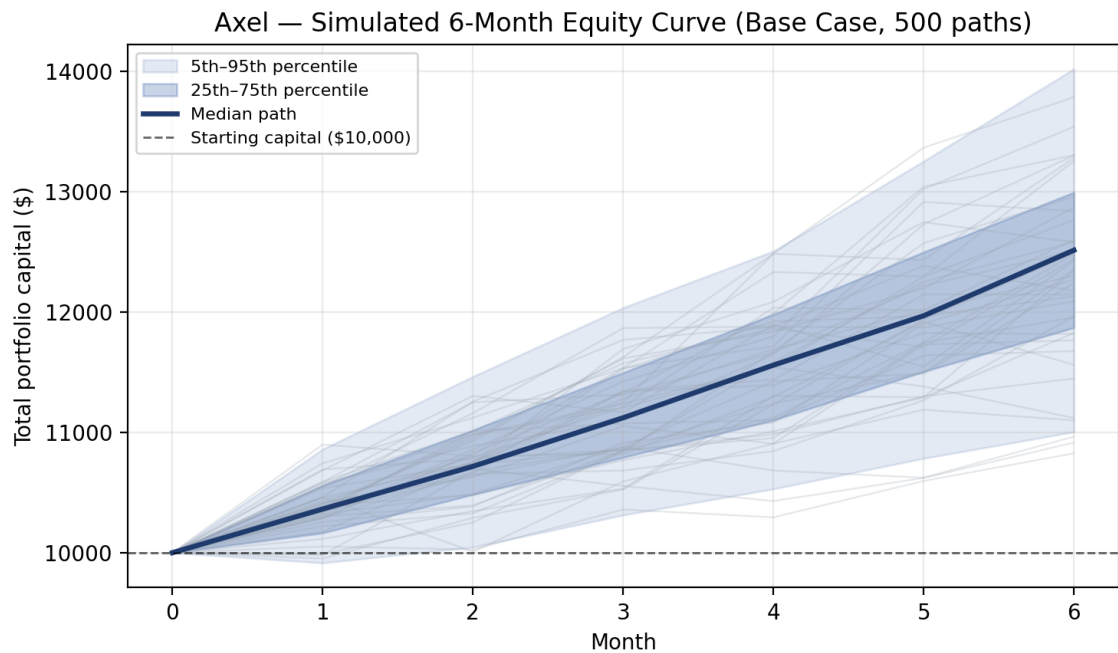


Figure 9.2: Simulated 6-month equity curve, Base case, 500 of 2,000 simulated paths shown for texture, with 25th–75th and 5th–95th percentile bands.

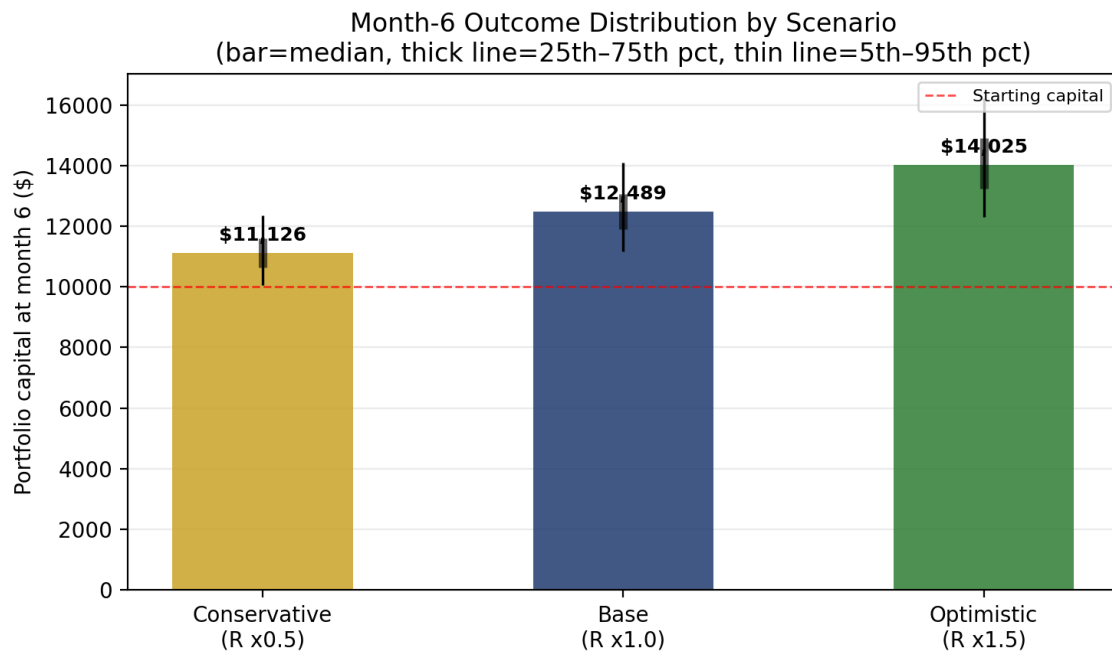


Figure 9.3: Distribution of month-6 outcomes across all three scenarios.

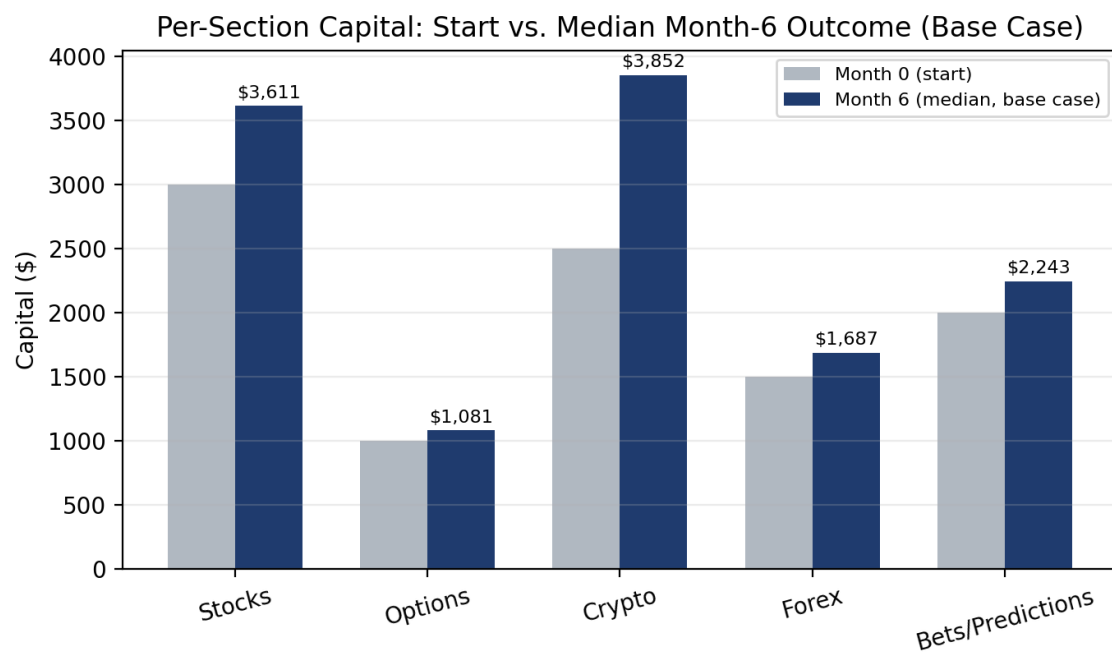


Figure 9.4: Median simulated outcome per section after 6 months, Base case, versus starting capital.

10. Technology Stack and Language Selection

10.1 Mapping languages to trust levels

Because the boundary between trust levels is a database record rather than a shared codebase (Chapter 2), different processes can genuinely use different languages without a painful integration cost.

Process	Language	Rationale
Agent service	Python	LangGraph, LLM SDKs, and the full ML ecosystem live here; no real substitute
Trading core (risk/, execution/)	Python now; Go as a later upgrade	Hypothesis property-testing — the project’s most valuable test suite — is Python-native. Not competing on HFT-grade latency per this system’s own scope
Watchdog	Go, when it graduates from stub	Deliberate isolation: a small, dependency-light, fast-starting binary decoupled from the Python process it watches is a correctness argument, not just a speed one
Data / ML layer	Python (pandas, scikit-learn, LightGBM/XGBoost; FinRL deferred)	Matches the explicit “defer RL, start with rules + gradient-boosted trees” decision in Chapter 7
Dashboard	Grafana first; custom Type-Script/React only if it falls short	Fastest path to a working operational view

10.2 On Java as an alternative

Java remains a legitimate alternative to Go for the Trading core specifically, with real precedent: **LMAX Exchange** runs its entire matching engine in Java, using the LMAX Disruptor (a lock-free ring buffer) to achieve microsecond-level latency — proof that JVM code can get very close to C++ once JIT-warmed in a long-running process. Java’s mature concurrency tooling (`java.util.concurrent`, virtual threads in recent JDKs) and strong typing make it defensible if the team already has strong Java fluency.

The tradeoff versus Go: JVM deployment is heavier (shipping a JVM plus a jar versus one static binary), GC pauses need tuning even if rare, and it fits this system’s Docker Compose plus small-VPS hosting default less cleanly. **Prefer Go unless existing team Java fluency outweighs that deployment simplicity** — and don’t adopt either until Phase 6 or later makes clear that Python’s throughput is the actual bottleneck, not a guess.

10.3 Reference open-source repositories

Structural references and component libraries to adapt — not endpoints to run unmodified.

Repository	Role	Fit
langchain-ai/ langgraph	Orchestration framework	Agent service’s core control-flow engine
virattt/ ai-hedge-fund	Manager/Analyst/Risk hierarchy reference	Structural reference for the Agent service
TauricResearch/ TradingAgents	Multi-agent debate pattern	Reference for the Expert Panel’s debate mechanism
AI4Finance- Foundation/FinRL	RL training framework	ML/signal layer, deferred until a simpler baseline is beaten
AI4Finance- Foundation/FinGPT	Financial-tuned LLM	Sentiment Analyst sub-agent
freqtrade/freqtrade	Crypto execution bot	Reference for Crypto section’s Execution Bot
hummingbot/ hummingbot	Market-making execution	Reference if a market-making strategy is added later
stefan-jansen/ machine-learning- for-trading	ML-for-trading reference code	Source material for the ML layer, not a runnable bot

11. Accounts, Capital, and Security

11.1 Account structure

Each section connects to its own funded account and its own API key — never a single shared credential across sections. A compromised or buggy key in one section cannot touch another section’s funds.

11.2 Security requirements (non-negotiable)

- API keys scoped to **trade-only, withdrawal-disabled** at the provider level, wherever supported
- Keys stored as environment variables (`.env`), never committed to source control; a secrets manager on the server
- Keys never exposed to the Agent service — only the Trading core holds them
- Two-factor authentication enabled on every underlying account; rotate keys on any suspicion
- Paper trading is the default and required first phase for every section; live capital is an explicit, manual, per-section decision
- Real-money rollout begins with a small, loss-tolerable amount per section

11.3 Self-custody wallets vs. exchange accounts (Crypto)

Exchange accounts with scoped, trade-only API keys are the recommended starting point. Self-custody wallets represent a materially larger attack surface for an automated system and are not recommended until the exchange-account approach has a proven operational track record.

12. Testing Strategy

12.1 Priority order

1. **Property-based tests for the risk package** (highest priority; the entire safety claim of this system depends on it). `tests/property/test_risk_invariants.py` uses Hypothesis to assert, across randomized inputs, that no approved order can ever exceed any configured limit. **Status: implemented, 4/4 passing.**
2. **Unit tests** for the loss manager’s flatten-policy dispatch and the order state machine’s illegal-transition rejection. **Status: implemented, passing.**
3. **Replay tests.** Feed a recorded market day through the whole pipeline using `core.clock.FakeClock` and a stubbed LLM layer; outputs must be bit-for-bit deterministic given the same inputs.
4. **Contract tests** for broker adapters against the paper API and a recorded-response fake.
5. **Chaos drills:** kill processes mid-order, drop the network, feed stale data, return malformed LLM output — the system must fail **closed** (no new orders) in every case.
6. **Adversarial prompt tests:** injection strings embedded in fake emails and headlines must not change any Proposal’s action or size.

Design Note

Verified in this build. Running the CI lint check with a deliberately introduced LLM import into `risk/sizer.py` produces exit code 1 and a clear failure message naming the offending file and import — confirming the enforcement mechanism actually works, not just that it exists in the design.

13. Phased Build Plan

Vertical build: one section fully proven before the next starts. Each phase has an explicit exit test; do not start the next phase until it passes. Estimates are ranges for one developer working part-time, not commitments.

Phase	Deliverables	Exit criteria	Est.
0. Foundations	Repo, Docker Compose (Postgres+TimescaleDB), migrations, typed config, structured logging, injectable clock, CI (lint/types/tests), secrets via <code>.env</code> , CI check that risk/execution import no LLM libraries	Fresh clone → one command → DB up, tests green	1 wk
1. Deterministic core	Contracts; risk rules, sizer, loss manager, allocator clamp; order state machine; Alpaca paper adapter; idempotent orders; reconciliation job; watchdog + kill switch; email alert on HALT. Driven by a dummy proposal script — NO LLM YET	Property tests pass. Kill-switch drill: simulate 10% drawdown, confirm HALT/cancels/alert/blocked resume. Paper orders round-trip with matching DB and broker state	3–4 wks
2. Data + backtester	Ingest Alpaca/Polygon bars, FRED, EDGAR, one news source, all point-in-time. Backtest engine with fees, slippage, calendar, walk-forward splits. Validation report. Two or three baseline strategies	Backtester reproduces a known result on a textbook strategy. A deliberately overfit strategy is REJECTED by the validation gate	3–4 wks
3. Agent layer (Stocks only)	Analysts emitting schema-validated signals; Expert Panel; Section Manager; Axel proposal logic; LLM budget guard; injection-resistant ingestion; event-driven triggers	End-to-end paper run for a week, zero schema violations, every proposal traceable, LLM spend under budget	3–4 wks
4. Dashboard + comms	Grafana on TimescaleDB: P&L, positions, drawdown, exposure, decision log. Email alerts, approval flow, read-only chat	You can answer “why did it take that trade?” from the dashboard alone	2 wks
5. Paper soak (Stocks)	Continuous paper mode; track calibration, slippage vs. backtest, incident log	Live-promotion gate (Chapter 14) met	8+ wks
6. Tiny live (Stocks)	Small, loss-tolerable capital; manual approval on every trade at first, relaxed gradually	Live results within tolerance of paper; no risk incidents	ongoing
7. Crypto	Exchange adapter, on-chain and funding-rate analysts, UTC day boundaries, no leverage. Reuses the core unchanged	Repeat Phase 5 exit criteria for Crypto	4–6 wks

Phase	Deliverables	Exit criteria	Est.
7b. Forex	Broker adapter (OANDA/IBKR), rate-differential and carry-trade analysts, FX-volatility early warning. Reuses the core unchanged	Repeat Phase 5 exit criteria for Forex	4–6 wks
8. Options	IV data source, Greeks, defined-risk structures only, assignment/expiry handling	Pricing model matches vendor Greeks within tolerance; expiry drills pass	6–8 wks
9. Bets/Predictions	Only after written legal review per venue. Calibration-first	Legal sign-off recorded; calibration beats market-implied baseline in paper	TBD
10. Quant Research Agent activation	Turn on observe → propose → backtest → PR loop for one proven section first	Human-review gate confirmed actually exercised, not rubber-stamped, before extending	after Ph. 6

Design Note

Forex placement. The original spec included Forex as a full section; it reuses the proven core exactly as Crypto and Options do, so it is slotted alongside Phase 7 rather than delaying the critical path — it can run in parallel with Crypto once Stocks has cleared Phase 6, or be deferred behind Options if bandwidth is limited.

Risk Note

The temptation at every phase boundary is to start the next section before the current one has actually cleared its exit criteria. Resist it. A single section proven end-to-end — including a real validated strategy and a real paper-trading track record — is worth more than five sections that all look complete but have never been tested against real data or real capital behavior.

14. Validation and Promotion Gates

The values below are defaults, fixed **before** looking at any results and recorded with a reason if changed later. Thresholds chosen after seeing results are a form of overfitting.

14.1 Strategy admission to the playbook

Test	Default
Data	Real historical data only, point-in-time, survivorship-bias-free where possible; costs and slippage included
Split	Walk-forward with an untouched final out-of-sample block; never tune on it
Sample size	At least 100 out-of-sample trades (more for low-frequency strategies)
Expectancy (net, R)	0.1R–0.5R per trade. Above ~1R sustained triggers a leakage audit
Risk-adjusted	Out-of-sample Sharpe above 0.7 net (default); max drawdown within the section’s tolerance
Robustness	±20% parameter perturbation does not flip the sign of expectancy; results hold across at least two market regimes
Multiple testing	Track how many variants were tried; apply a deflated-Sharpe adjustment

14.2 Validating the LLM layer

- Never backtest LLM judgment against a period the model may have seen in training (Chapter 8).
- Run a rules-only baseline in parallel; if the LLM Panel does not beat it after costs, simplify.
- Track calibration of panel confidence (Brier score) and disagreement between analysts.

14.3 Paper-to-live gate (per section)

- At least 8 weeks and 200 paper trades (or an agreed equivalent for low-frequency sections)
- Paper expectancy and slippage within tolerance of backtest assumptions
- Zero unresolved reconciliation breaks; zero risk-rule bypass incidents; kill-switch drill passed in the last 30 days

- Recovery drill passed: kill the process mid-order, restart, and state reconciles correctly
- Explicit manual sign-off, logged. Start with small capital and per-trade human approval

15. Top Risks and Mitigations

Risk	Mitigation
Overfitting / leakage produces fake edge	Strict validation gate, untouched OOS block, multiple-testing adjustment, LLM contamination awareness (Chapter 8)
Scope creep across five sections	Vertical slice; explicit phase exit criteria; later sections reuse the proven core
Runaway LLM cost	Budget guard (<code>agents/llm_budget.py</code>), event-driven triggers, model tiering
Silent failure (stale data, drifted state)	Reconciliation (<code>execution/reconcile.py</code>), heartbeats, fail-closed defaults
LLM-driven bad trade	LLM cannot reach execution (CI-enforced); deterministic veto; per-trade risk cap; human approval in live mode
Legal/access issues on venues	Confirm per venue before building; paper trading first

16. Summary: Every Correction from v1 to v2

#	v1 design	Problem	v2 fix
1	Axel = "LLM + deterministic hybrid," also the "final trade gate"	An LLM in the last gate breaks the core reasoning/execution separation	Axel proposes; a deterministic Risk package disposes (Chapter 2)
2	"Kelly capped at 25%" vs. "max position 5%"	Ambiguous: 25% of what? If 25% of capital, the 5% cap always wins and Kelly math is theater	Defined as $0.25 \times \text{full Kelly } f^*$, then <code>min()</code> against both a notional cap and a risk-at-stop cap (Chapter 6)
3	Position cap was purely notional	A 5% notional position with a wide stop can lose far more than intended	Added a risk-at-stop cap (0.5–1% of section capital), computed from actual stop distance
4	Loss Manager, Risk Engine, Profit Optimizer, Position Sizer named but absent from the architecture	Unowned enforcement means gaps or duplicated logic	All four are real, tested modules inside one <code>risk/</code> package
5	Kill-switch said "pause" with no defined semantics	Open options and 24/7 crypto keep moving during an undefined pause	Peak-to-trough vs. high-water mark; independent Watchdog process; explicit per-section flatten policy
6	Panel confidence threshold of 0.3	Verbalized LLM confidence is uncalibrated; the number may be meaningless	Treated as a placeholder pending real calibration tracking (Brier score) against paper outcomes
7	"Backtested and scored" with no pass/fail criteria	Anything could pass a vague bar	Fixed thresholds committed before results are seen: 100+ OOS trades, 0.1–0.5R, Sharpe ≥ 0.7 , $\pm 20\%$ robustness
8	Inbound email/news/social feeds LLM analysts directly	Prompt injection risk: crafted content could steer an analyst	All ingested text is untrusted data, never instructions; analysts have no side-effect tools

#	v1 design	Problem	v2 fix
9	No human-approval component, despite being required by the non-goals	Nothing implemented the promised gate	<code>execution/approval.py</code> : every live-mode proposal waits for approval and expires if not acted on
10	No operational hardening specified	24/7 systems fail in boring ways: stale data, duplicate orders, drift	Idempotent order IDs, reconciliation, order state machine, heartbeats, append-only audit tables
11	Four-plus sections built in parallel	Different data/brokers/math/legal first, end-to-end, before any risk per section stalls everything	Strictly vertical build: Stocks first, end-to-end, before any other section starts
12	LLM strategies validated the same way as deterministic strategies	An LLM can have memorized the backtest period, contaminating any backtest of its own judgment	Split validation paths: backtest for deterministic rules, forward paper trading + calibration for LLM judgment

Design Note

Twelve corrections, all now reflected in the architecture, the risk math, the phase plan, and — for the risk package specifically — in working, tested code. This is the version to build from.

This document describes software engineering structure only. It is not financial or legal advice, and automated trading can lose money even when validated.